Rossmann Store Sales Prediction Report

Prepared By – Ayeshkant Ray

Course – SparkIIT Data Science

Date – April 2026

Tools – Python (numpy, pandas, seaborn, matplotlib, sklearn)

1. Abstract

This report presents a machine learning solution for predicting daily sales figures across Rossmann drug stores in Germany. The dataset, sourced from Kaggle's Rossmann Store Sales competition, contains over one million sales records from 1,115 stores, merged with store-level metadata. A Linear Regression model was trained to forecast sales based on temporal features, promotional activity, store type, assortment level, competition proximity, and holiday indicators.

Exploratory data analysis revealed that store type 'b' and assortment type 'b' (extra) yield the highest average sales; short-term promotions (Promo) drive a clear uplift in sales while continuous promotions (Promo2) show diminishing returns; and sales peak during November–December due to seasonal shopping patterns. A time-based 80/20 train-test split was employed to prevent future data leakage, simulating a real forecasting scenario.

Given the significant non-linear and interaction effects in the data, the model's R2 score of 0.226, which explains roughly 22.6% of the sales variance, is a small but expected baseline. The model's shortcomings are validated by the residual and actual-vs-predicted plots, which clearly support the use of ensemble techniques like Random Forest or XGBoost as a next step.

2. Introduction

One of the main issues in retail analytics is sales forecasting, which directly affects revenue planning, staffing, and inventory management. Accurately forecasting future shop sales necessitates capturing a complex combination of competitive dynamics, regional considerations, promotional effects, and temporal trends. The Rossmann Store Sales dataset provides a rich real-world context for this issue; it was first made available as part of a Kaggle competition.

Rossmann operates over 3,000 drug stores across Europe. The task here is to predict daily sales figures (Sales) for a subset of 1,115 German stores. The dataset comprises two files: train.csv containing transactional sales records (date, store, promotions, holiday flags, open/closed status, and customer counts), and store.csv containing store-level attributes (store type, assortment category, competition distance, and promotional programme details).

For this project, the baseline model is linear regression. It establishes a performance floor and determines which features contain predictive signal, laying the groundwork for more complex modeling in later cycles, even though it is unlikely to fully represent the complexity of sales dynamics.

3. Objectives

The primary objectives of this project are as follows:

- To merge and explore the Rossmann transactional and store metadata datasets, identifying key patterns in sales distributions, store types, promotional effectiveness, and seasonality.
- To handle missing values in competition and promotional columns appropriately, based on domain context rather than generic imputation.
- To filter and preprocess the data — removing closed-store days and leakage-prone columns — to ensure only meaningful training signal is retained.
- To engineer temporal features (Year, Month, DayOfWeek, WeekOfYear) from the raw date column to give the model time-awareness.
- To encode categorical variables using One-Hot Encoding, avoiding the false ordinal relationships that Label Encoding would impose on nominal store attributes.
- To perform a time-based train-test split (80/20) to simulate a genuine sales forecasting scenario without leaking future data into training.
- To train a Linear Regression model and evaluate it using R^2 Score and Root Mean Squared Error (RMSE), supported by Actual vs Predicted and Residual diagnostic plots.
- To critically interpret the results and outline the performance ceiling of linear modelling on non-linear retail data, motivating a roadmap to ensemble methods.

4. Literature Review / Existing Work

Retail sales forecasting has evolved significantly from traditional statistical time-series models like ARIMA and Exponential Smoothing, which are effective but limited to univariate data. The rise of large retail datasets has led to increased use of machine learning techniques. Early models, such as Linear Regression, provided low predictive performance, prompting a shift to more sophisticated methods. The Rossmann Store Sales Kaggle competition highlighted the effectiveness of gradient boosting methods like XGBoost, achieving accurate predictions with RMSPE under 10%. Ensemble techniques, including Random Forest and LightGBM, dominate due to their ability to model complex relationships and manage varying data types. Improved predictive performance is largely attributed to effective feature engineering, which includes extracting temporal attributes and incorporating domain-specific variables. Despite progress, linear models still struggle with interaction effects and sudden demand spikes, reinforcing the preference for non-linear ensemble methods in contemporary retail forecasting.

5. Dataset Description

The dataset used in this project was sourced from the Kaggle Rossmann Store Sales competition and consists of two files:

5.1 train.csv

Contains 1,017,209 rows of daily sales records with the following fields:

- Store — unique store identifier (1 to 1,115)
- DayOfWeek — day of the week (1 = Monday, 7 = Sunday)
- Date — calendar date in YYYY-MM-DD format
- Sales — daily turnover in euros (target variable)
- Customers — number of customers visiting that day (excluded at modelling stage to prevent leakage)
- Open — binary flag indicating whether the store was open (1) or closed (0)
- Promo — binary flag indicating whether the store was running a short-term promotion
- StateHoliday — categorical flag for state holidays (a = public, b = Easter, c = Christmas, 0 = none)
- SchoolHoliday — binary flag indicating whether local schools were on holiday

5.2 store.csv

Contains metadata for each of the 1,115 stores:

- StoreType — store format category (a, b, c, d)
- Assortment — product range level (a = basic, b = extra, c = extended)
- CompetitionDistance — distance in metres to the nearest competitor
- CompetitionOpenSinceMonth / Year — when the nearest competitor opened
- Promo2 — binary flag for whether the store participates in a continuous promotion
- Promo2SinceWeek / Year — when the continuous promotion began
- PromoInterval — months in which Promo2 is active (e.g. 'Jan,Apr,Jul,Oct')

6. Methodology

The project follows a standard supervised machine learning pipeline across eight stages:

6.1 Data Loading and Merging

The two files were loaded using pandas and merged on the Store column using an inner join, producing a unified dataset of 1,017,209 rows and 18 columns. Initial inspection via df.head(), df.info(), and df.describe() confirmed data types, ranges, and the presence of null values in several columns.

6.2 Null Value Handling

Missing values were identified and handled column by column based on domain context:

- CompetitionDistance: Filled with the column median (~2,325 m), as the distribution is right-skewed and the median is more robust than the mean to outlier stores with unusually distant competitors.
- CompetitionOpenSinceMonth and CompetitionOpenSinceYear: Filled with 0, indicating no recorded competition opening — interpreted as competition not yet established.
- Promo2SinceWeek and Promo2SinceYear: Filled with 0, as missing values indicate the store does not participate in the continuous promotion programme.
- PromoInterval: Filled with 'None', distinguishing non-participating stores from those with a defined promotional calendar.

6.3 Exploratory Data Analysis

EDA was conducted through a series of targeted visualisations, each designed to answer a specific analytical question about the data. Closed-store days (Sales = 0) were excluded from all EDA visualisations, as they do not represent real trading activity and would distort averages.

6.4 Feature Engineering

Three time-based features were extracted from the Date column: Year, Month, and WeekOfYear. DayOfWeek was already present in the dataset. These features allow the model to capture seasonal and cyclical patterns without using the raw date string, which is non-numeric.

6.5 Categorical Encoding

One-Hot Encoding was applied to StoreType, Assortment, StateHoliday, and PromoInterval using pandas get_dummies with drop_first=True to avoid multicollinearity. Label Encoding was deliberately not used, as these are nominal categories with no inherent ordinal relationship — encoding them as integers would imply a false ranking to the model.

6.6 Train-Test Split (Time-Based)

Because the data is temporal in nature, a random split would allow the model to learn from future data when predicting the past — a form of data leakage. Instead, the dataset was sorted by date and split at the 80th percentile of time: the earliest 80% of dates formed the training set and the latest 20% formed the test set. This simulates a genuine forecasting scenario where the model predicts sales for future dates it has never seen.

6.7 Model Training

A Linear Regression model from scikit-learn was instantiated with default hyperparameters and fitted on the training features. Predictions were then generated on the held-out test set.

6.8 Evaluation

Model performance was assessed using two metrics — R^2 Score and Root Mean Squared Error (RMSE) — supplemented by two diagnostic plots: an Actual vs Predicted scatter plot and a Residual plot.

7. Exploratory Data Analysis — Key Findings

7.1 Sales Distribution

When filtered to open-store days only, daily sales follow an approximately normal distribution centred around €5,000–6,000 per day. Including closed days (Sales = 0) introduces a mass at zero that artificially suppresses averages — this is why all EDA uses the filtered dataset.

→ All EDA visualisations use Sales > 0 filter. Including closed days would skew distributions and mislead interpretation of average store performance.

7.2 Top and Bottom Stores

The top 10 stores by average daily sales significantly outperform the bottom 10, with a gap of several thousand euros. This high inter-store variance suggests that store-level fixed effects are an important driver of sales — a factor that linear regression partially captures through store-type and assortment features, but that store-ID embeddings or fixed-effect models would capture more completely.

7.3 Store Type and Assortment

Store Type 'b' shows markedly higher average sales than types 'a', 'c', and 'd'. Assortment Type 'b' (extra) outperforms both the basic ('a') and extended ('c') assortment levels, suggesting that a targeted extra assortment — rather than the broadest product range — is most strongly associated with high sales.

→ Store Type 'b' and Assortment 'b' (extra) are the highest-performing configurations. These categorical features carry significant predictive signal for the model.

7.4 Effect of Promotions

Short-term promotions (Promo = 1) show a clear and consistent uplift in daily sales compared to non-promotional days. Continuous promotions (Promo2), however, show a weaker or slightly negative association with sales. This counter-intuitive result is plausibly explained by customer habituation: if discounts are permanent, they lose their urgency effect and may attract price-sensitive customers who spend less on non-discounted items.

→ Promo (short-term) is a strong positive predictor of sales. Promo2 (continuous) shows diminishing returns — possibly because ongoing discounts reduce perceived value and purchase urgency.

7.5 Sales by Day of Week and Month

Sales peak mid-week and are notably lower on Sunday (DayOfWeek = 7), consistent with German retail regulations that restrict Sunday trading hours. Monthly analysis reveals a strong November–December spike driven by holiday-season shopping, and a mild dip in July that aligns with summer school holidays in Germany.

7.6 Holiday Effects

School holidays show minimal impact on sales. State holidays, by contrast, are associated with a noticeable decline — most stores operate with reduced hours or are fully closed on public holidays, suppressing daily totals.

7.7 Competition Distance

After binning CompetitionDistance into deciles, no strong monotonic trend emerges between proximity to competitors and sales. Stores very close to competitors do not consistently underperform stores with distant competitors — likely reflecting the role of brand loyalty, store heritage, and location quality in sustaining sales regardless of nearby competition.

→ Competition proximity does not strongly predict sales in this dataset. Established stores appear to maintain loyal customer bases regardless of competitor distance.

8.Code

8.1 Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_squared_error, r2_score
```

8.2 Loading and Merging the Dataset

```
train = pd.read_csv("../Datasets/rossmann-store-sales/train.csv", low_memory=False)
store = pd.read_csv("../Datasets/rossmann-store-sales/store.csv")
print("Train shape:", train.shape)
train.head()
print("Store shape:", store.shape)
store.head()
df = pd.merge(train, store, on="Store")

df.shape
```

8.3 Exploratory Data Analysis (EDA)

```
df.head()
df.shape
df.info()
df.describe()
df.columns
```

8.3 Null Value Handling

```
df.isnull().sum()

df["CompetitionDistance"].median()
df["CompetitionDistance"] =
df["CompetitionDistance"].fillna(df["CompetitionDistance"].median())
df["CompetitionOpenSinceMonth"] = df["CompetitionOpenSinceMonth"].fillna(0)
df["CompetitionOpenSinceYear"] = df["CompetitionOpenSinceYear"].fillna(0)
df["Promo2SinceWeek"] = df["Promo2SinceWeek"].fillna(0)
df["Promo2SinceYear"] = df["Promo2SinceYear"].fillna(0)
df["PromoInterval"].unique()
df["PromoInterval"] = df["PromoInterval"].fillna("None")
df.isnull().sum()
```

8.4 Sales Distribution – Histogram

```
# How are Sales spread
data = df[df["Sales"] > 0]
sns.histplot(data=data, x="Sales", bins=100)
plt.show()
```

8.5 Top & Bottom 10 Stores by Avg Sales

```
filtered = df[df["Sales"] > 0]
store_avg=filtered.groupby("Store")["Sales"].mean().sort_values(ascending=False).reset_inde
x()
top10 = store_avg.head(10).copy()
bottom10 = store_avg.tail(10).copy()
top10["Group"] = "Top 10"
bottom10["Group"] = "Bottom 10"
combined = pd.concat([top10, bottom10])
palette = {"Top 10": "green", "Bottom 10": "red"}
sns.barplot(x="Store", y="Sales", data=combined, hue="Group", palette=palette,
dodge=False)
plt.title("Top 10 vs Bottom 10 Stores by Average Sales")
plt.xlabel("Store ID")
plt.ylabel("Average Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

8.6 Effect of Closed Days on Sales - Boxplot

```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
data_all=df.groupby("Store")["Sales"].mean().sort_values(ascending=False).head(10).index
sns.boxplot(x="Store", y="Sales", data=df[df["Store"].isin(data_all)], ax=axes[0])
axes[0].set_title("Including Closed Days (Sales = 0)")
data_filtered=filtered.groupby("Store")["Sales"].mean().sort_values(ascending=False).head(10
).index
sns.boxplot(x="Store", y="Sales", data=filtered[filtered["Store"].isin(data_filtered)],
ax=axes[1])
axes[1].set_title("Open Days Only (Sales > 0)")
plt.suptitle("Top 10 Stores — Effect of Including vs Excluding Closed Days", fontsize=13)
plt.tight_layout()
plt.show()
```

8.7 Effect of Store Type and Assortment on Sales

```
fig, axes = plt.subplots(1, 2, figsize=(16,6))
data_all = df.groupby("StoreType")["Sales"].mean().sort_values().reset_index()
sns.barplot(x="StoreType", y="Sales", data=data_all, ax=axes[0])
```

```
axes[0].set_title("Including Closed Days (Sales = 0)")
data_filtered = filtered.groupby("StoreType")["Sales"].mean().sort_values().reset_index()
sns.barplot(x="StoreType", y="Sales", data=data_filtered, ax=axes[1])
axes[1].set_title("Open Days Only (Sales > 0)")
plt.suptitle("Store Type vs Sales - Effect of Including vs Excluding Closed Days", fontsize=13)
plt.tight_layout()
plt.show()


fig, axes = plt.subplots(1, 2, figsize=(16,6))
data_all=df.groupby("Assortment")["Sales"].mean().sort_values(ascending=False).reset_index
()
sns.barplot(x="Assortment", y="Sales", data=data_all, ax=axes[0])
axes[0].set_title("Including Closed Days (Sales = 0)")
data_filtered=filtered.groupby("Assortment")["Sales"].mean().sort_values(ascending=False).r
eset_index()
sns.barplot(x="Assortment", y="Sales", data=data_filtered, ax=axes[1])
axes[1].set_title("Including Open Days Only (Sales > 0)")
plt.suptitle("Assortment vs Sales - Effect of Including vs Excluding Closed Days", fontsize=13)
plt.tight_layout()
plt.show()
```

8.8 Effect of Promotions on Sales

```
sns.barplot(x="Promo", y="Sales", data=filtered)
plt.title("Sales - With vs Without Promo")
plt.show()

sns.barplot(x="Promo2", y="Sales", data=filtered)
plt.title("Sales - With vs Without Promo2")
plt.show()
```

8.9 Sales by Days of Week and Month

```
sns.barplot(x="DayOfWeek", y="Sales", data=filtered)
plt.title("Average Sales by Day of Week")
plt.xlabel("Day (1-Mon, 7-Sun)")
plt.show()


df["Date"] = pd.to_datetime(df["Date"])
df["Month"] = df["Date"].dt.month
filtered = df[df["Sales"] > 0]  # recreate filtered AFTER datetime conversion refresh filtered
after adding Month column
sns.barplot(x="Month", y="Sales", data=filtered)
```

```
plt.title("Average Sales by Month")
plt.show()
```

8.10 Effect of Holiday Type on Sales

```
sns.barplot(x="SchoolHoliday", y="Sales", data=filtered)
plt.show()

sns.barplot(x="StateHoliday", y="Sales", data=filtered)
plt.show()
```

8.11 Effect of Competition Distance on Sales

```
sns.barplot(x="CompetitionDistance", y="Sales", data=filtered)
plt.show()

comp_data = filtered.copy()
comp_data["CompetitionDistance"] = pd.cut(comp_data['CompetitionDistance'], bins=10)
sns.barplot(x="CompetitionDistance", y="Sales", data=comp_data)
plt.xticks(rotation=45)
plt.show()
```

8.12 Preprocessing and Feature Scaling

```
df = df[df["Open"] == 1].copy() # Keeping stores which are open
df.drop(columns=['Open', 'Customers'], inplace=True) # Dropping Open as only 1 exist

df["Year"] = df["Date"].dt.year
df["WeekOfYear"] = df["Date"].dt.isocalendar().week.astype(int)
```

8.13 One-Hot Encoding

```
cols = ["StoreType", "Assortment", "StateHoliday", "PromoInterval"]
df = pd.get_dummies(df, columns=cols, drop_first=True)
```

8.14 Time Based Split

```
df = df.sort_values(by="Date")
split_index = int(len(df) * 0.8)
train_df = df.iloc[:split_index]
test_df = df.iloc[split_index:]
X_train = train_df.drop(columns="Sales")
y_train = train_df["Sales"]
X_test = test_df.drop(columns="Sales")
y_test = test_df["Sales"]

X_train = X_train.drop(columns="Date")
X_test = X_test.drop(columns="Date")
```

8.15 Model Training

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

8.16 Model Evaluation

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"Root Mean Squared Error:  {rmse:.4f}")
print(f"R2 Score: {r2:.4f}")

# Actual vs Predicted Sales Plot

plt.scatter(y_test, y_pred, alpha=0.3)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")
plt.show()

# Residual Sales Plot

residue = y_test - y_pred
plt.scatter(y_pred, residue, alpha=0.3)
plt.axhline(y=0, color='red')
plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.show()
```

9.Output Snapshots

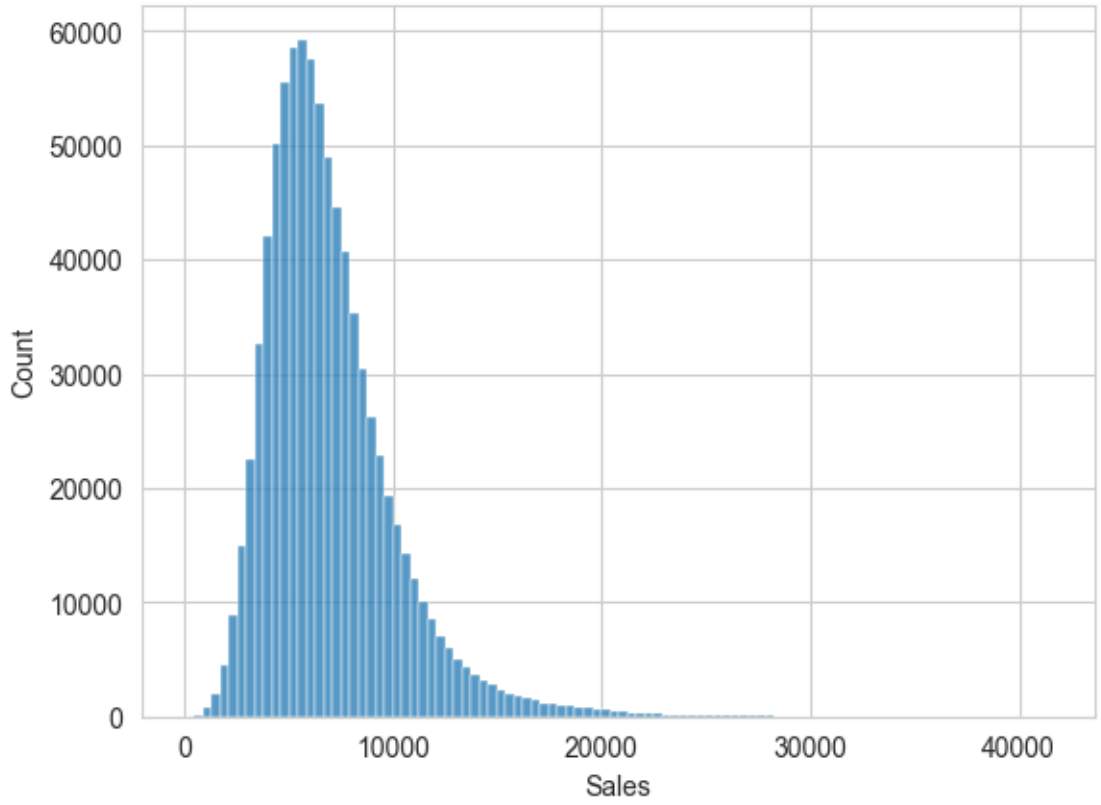

Figure 1

This histogram displays the frequency distribution of daily sales across all open store-days. The distribution is approximately bell-shaped and centred around €5,000–6,000, with a right-skewed tail representing high-performing days. Closed-store days (Sales = 0) are excluded, ensuring the distribution reflects genuine trading activity.

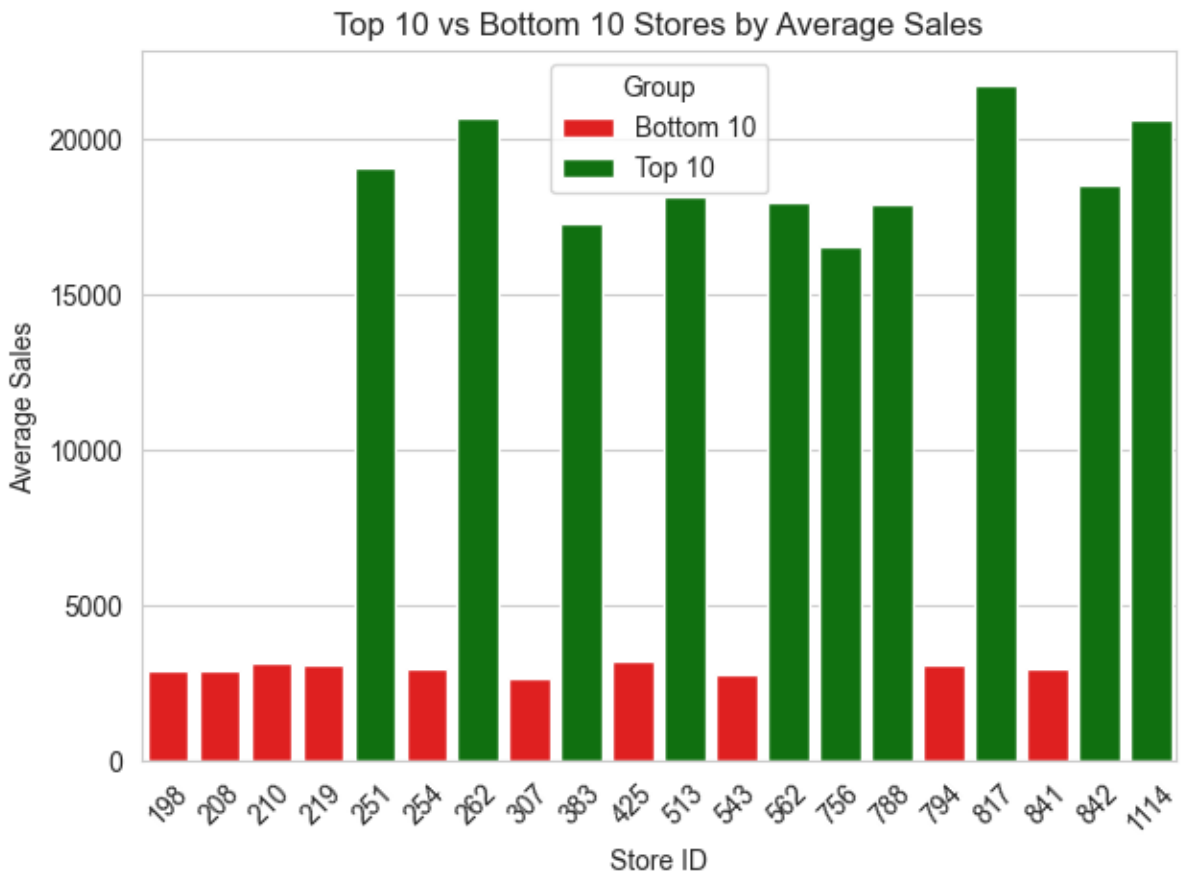

Figure 2

This grouped bar chart compares the ten highest and ten lowest performing stores by mean daily sales. Top-performing stores (shown in green) record average sales several thousand euros above the bottom-performing stores (shown in red), illustrating the high inter-store variance present in the dataset.

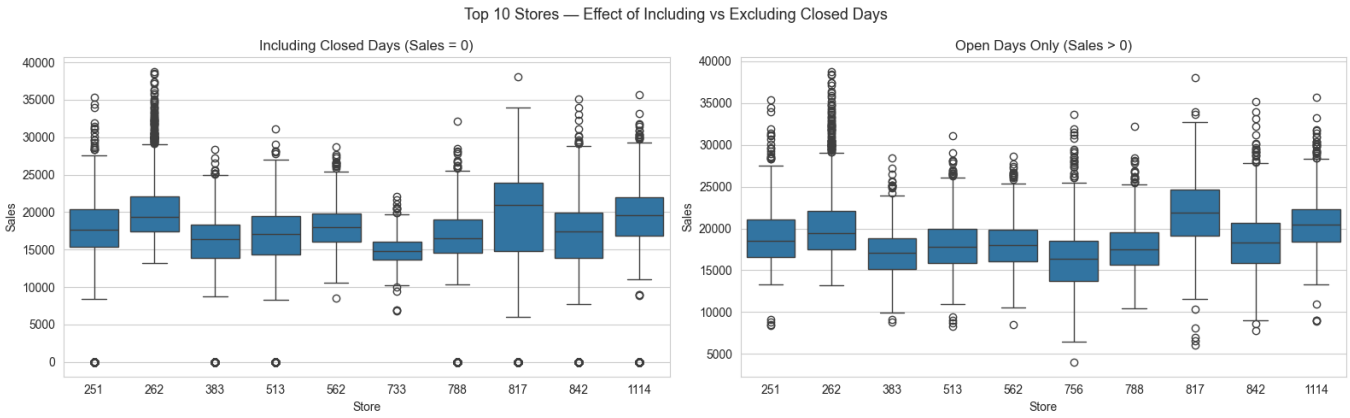

Figure 3

This side-by-side boxplot shows the top 10 stores' sales distributions with closed days included (left panel). The inclusion of zero-sales days compresses the interquartile range and pulls medians downward, demonstrating how closed-day records distort the apparent sales profile of a store.

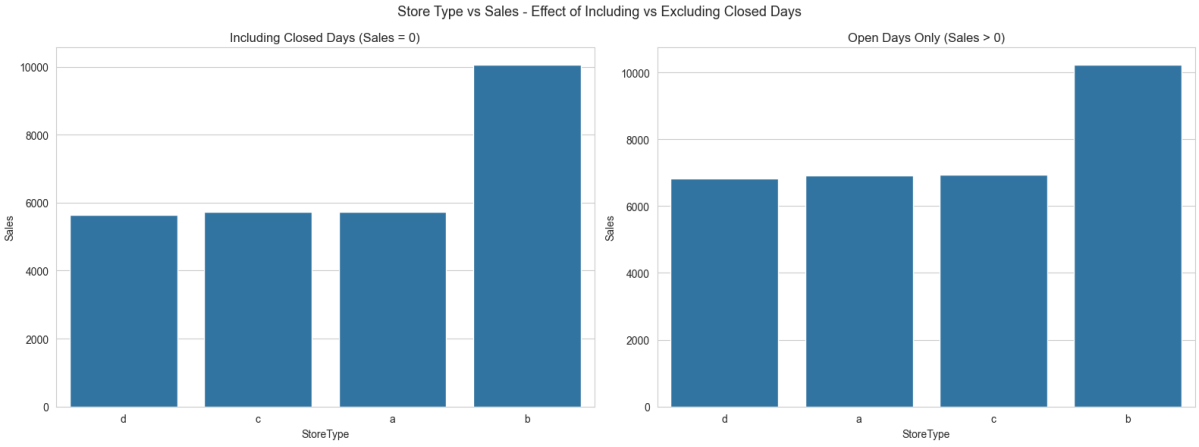

Figure 4

This dual bar chart compares average sales by store type (a, b, c, d) under both filtering conditions. Store Type 'b' consistently records the highest average daily sales. The left panel (all days) suppresses the differences, while the right panel (open days only) clearly reveals the performance gap between store types.

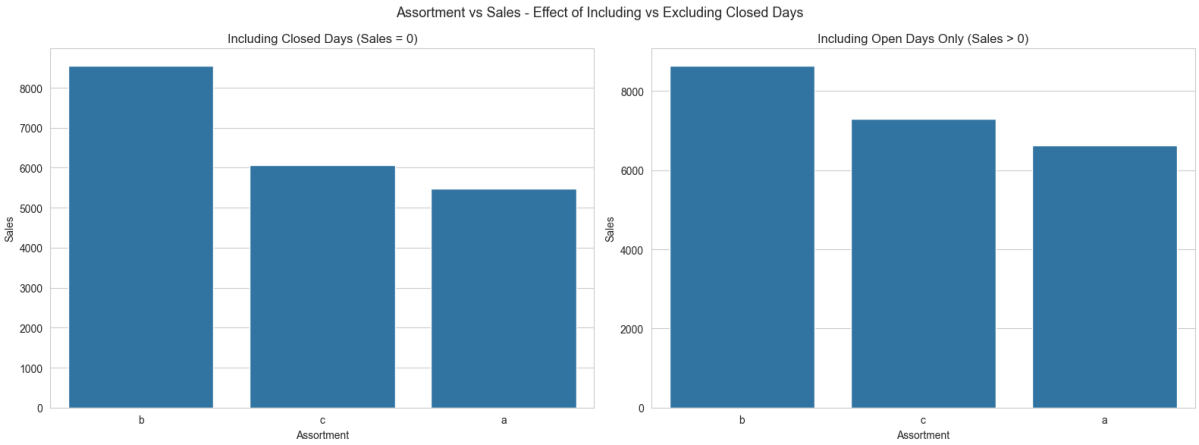

Figure 5

Similar in structure to Figure 5, this chart examines the relationship between assortment level and average sales. Assortment Type 'b' (extra) outperforms both the basic ('a') and extended ('c') assortments on open days, suggesting that a targeted extra product range is more commercially effective than the broadest selection.

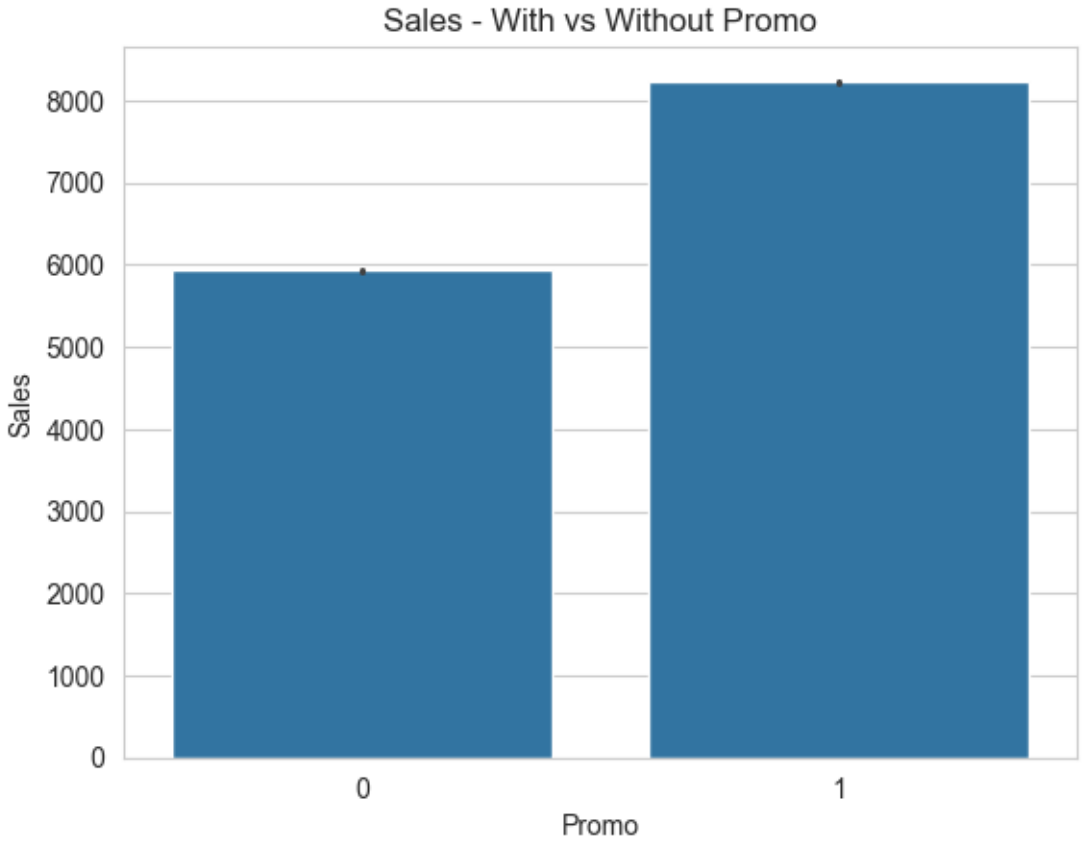

Figure 6

This bar chart compares average daily sales on promotional versus non-promotional days. Stores running a short-term promotion (Promo = 1) show a clear and consistent

sales uplift, confirming that Promo is one of the strongest positive predictors of daily revenue in the dataset.

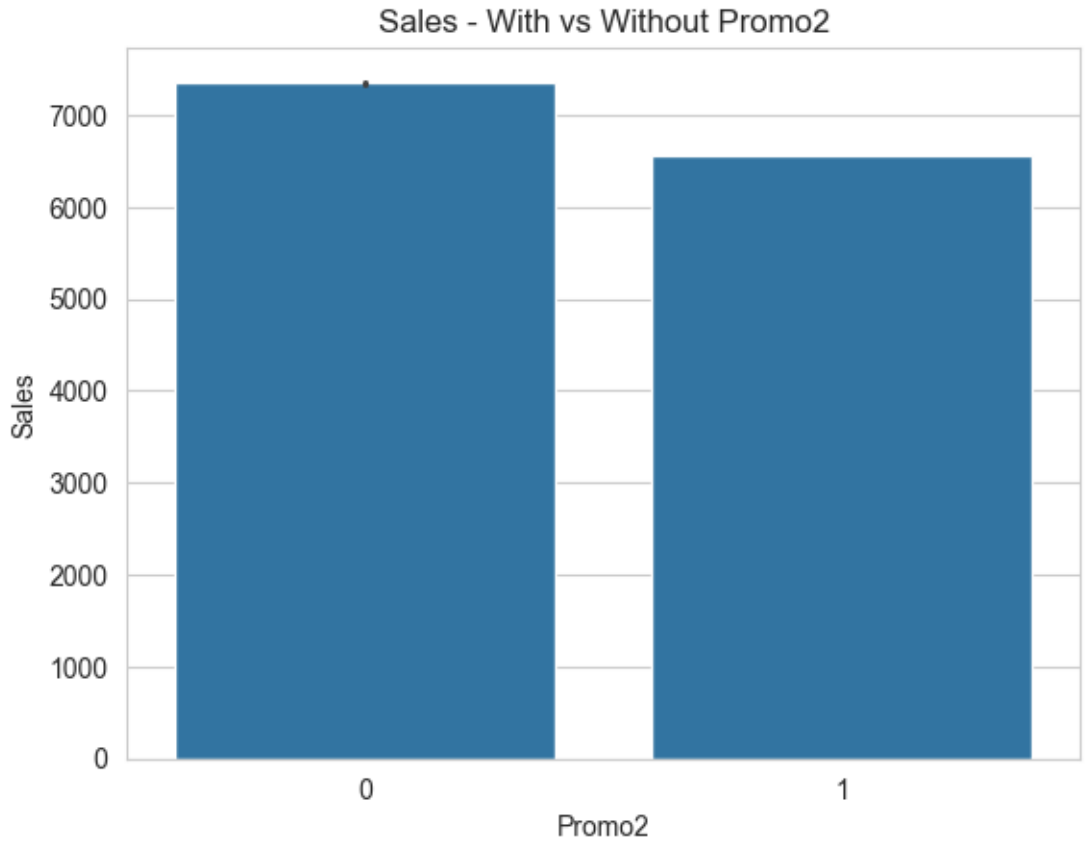

Figure 7

Mirroring Figure 6 but for the continuous promotion flag (Promo2), this chart reveals a weaker or marginal association between Promo2 participation and higher sales. The diminishing effect is consistent with the hypothesis that permanent promotions lose their urgency signal over time.

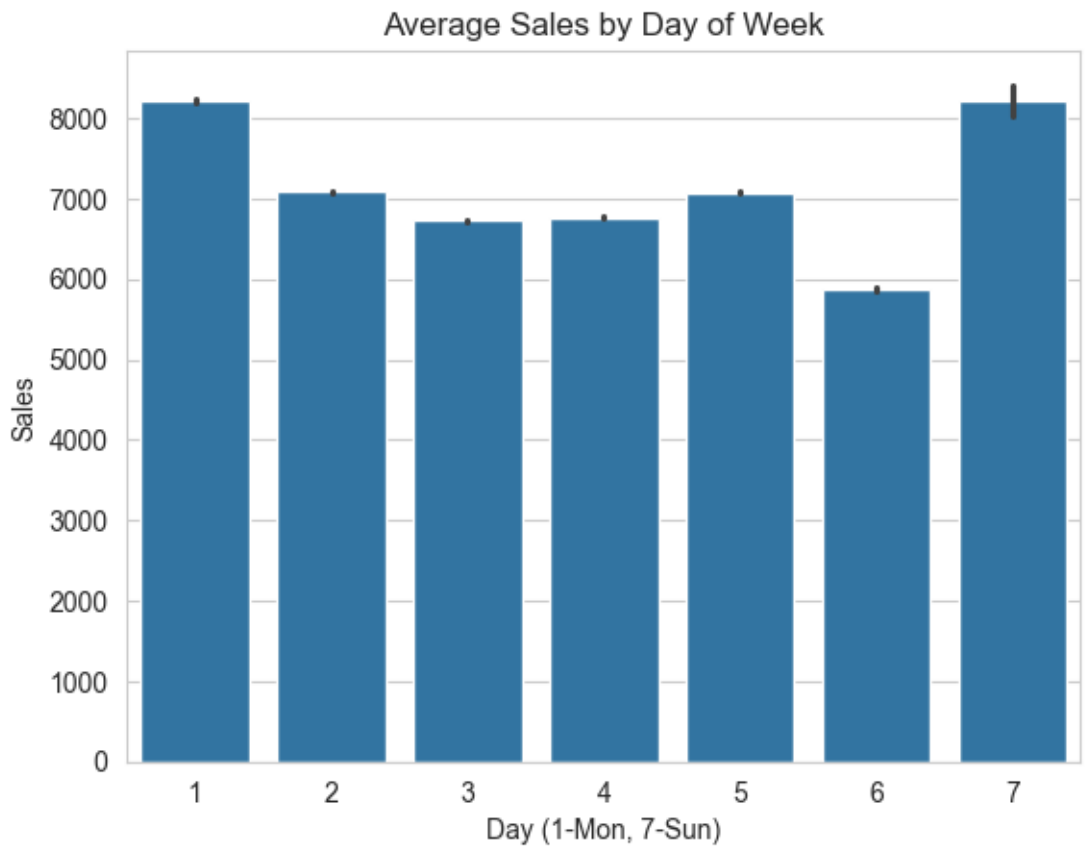

Figure 8

This bar chart plots mean daily sales against each day of the week (1 = Monday through 7 = Sunday). A clear mid-week peak is visible, with sales declining toward the weekend. Sunday (Day 7) records the lowest average, consistent with German retail regulations restricting Sunday trading hours.

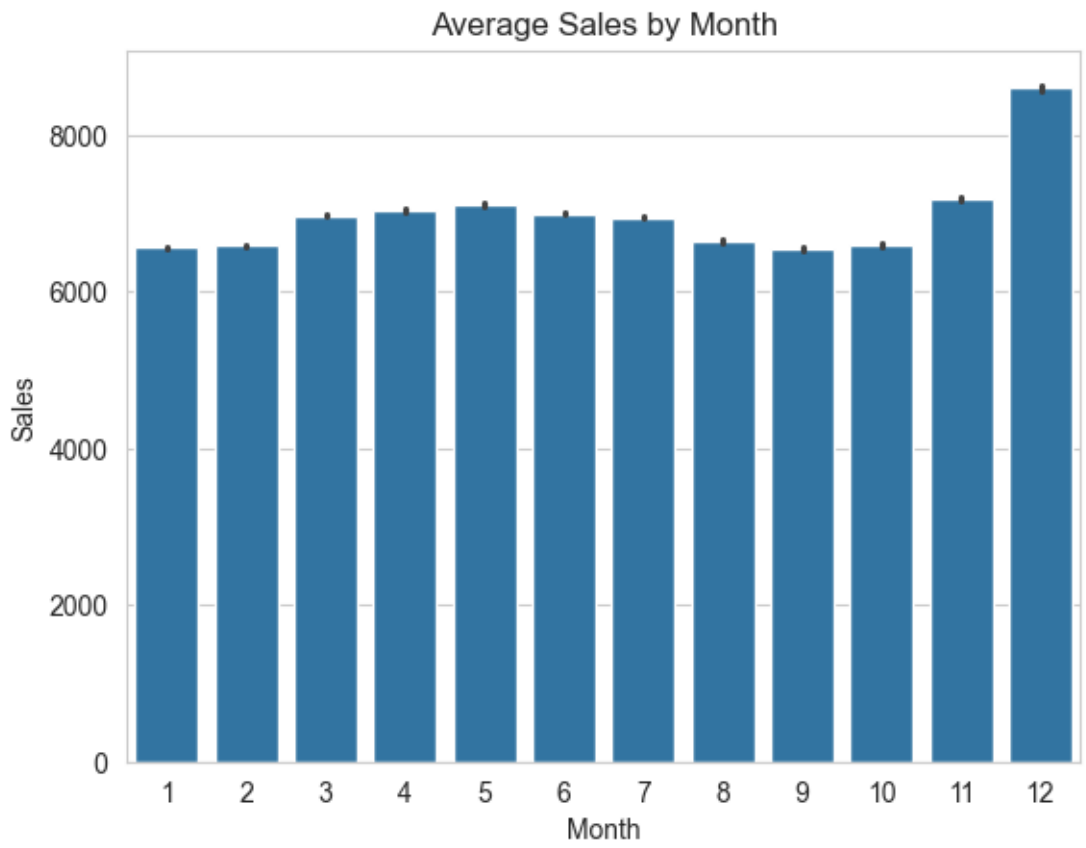

Figure 9

This monthly bar chart reveals the strong seasonal pattern in the dataset. Sales rise steadily toward a clear November–December peak driven by holiday-season shopping. A mild dip is visible around July, aligning with German summer school holidays and reduced consumer footfall.

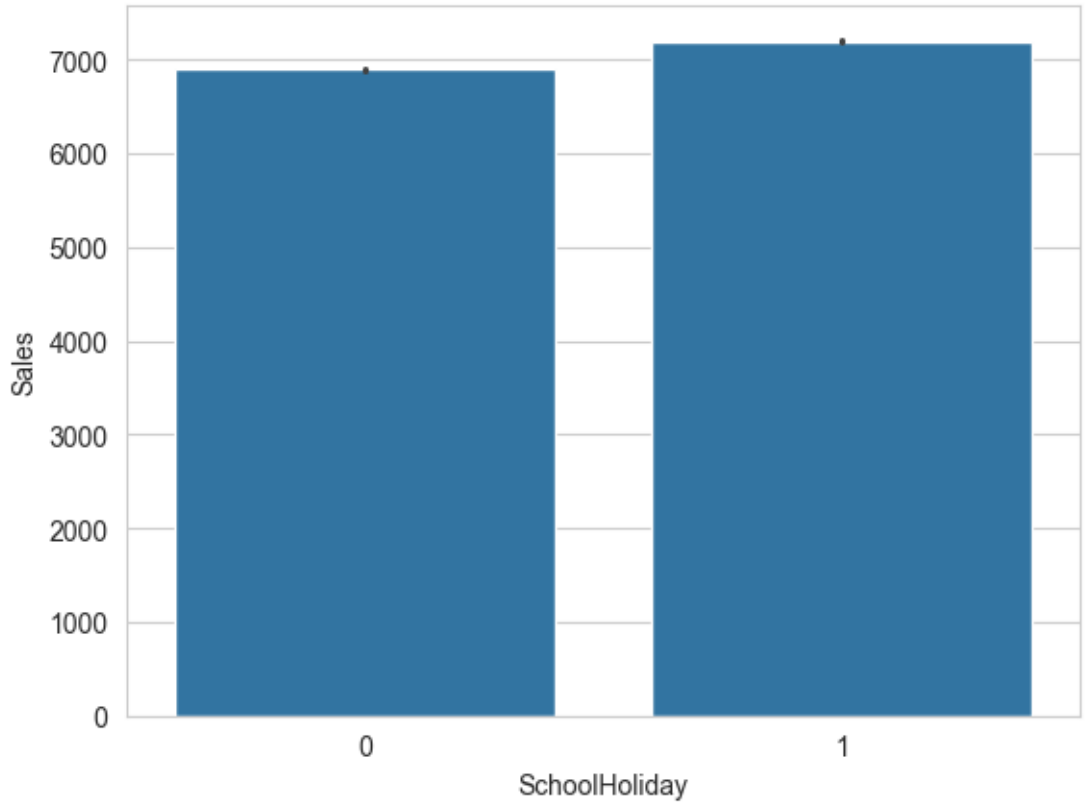

Figure 10

This bar chart compares average sales on school holiday versus non-school-holiday days. The difference is minimal, indicating that school holidays do not exert a significant independent effect on daily store turnover in this dataset.

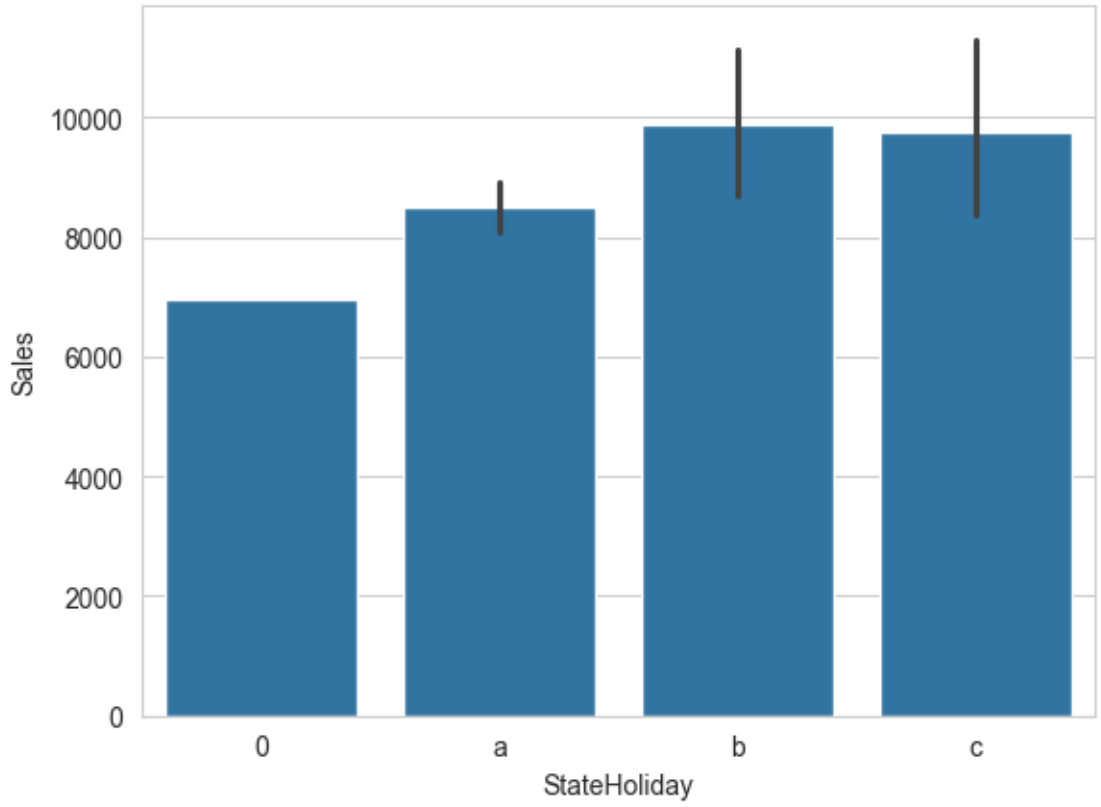

Figure 11

In contrast to Figure 10, this chart shows that state holidays (public, Easter, Christmas) are associated with noticeably lower average sales. This aligns with the fact that many stores operate with reduced hours or remain fully closed on public holidays, suppressing daily sales totals.

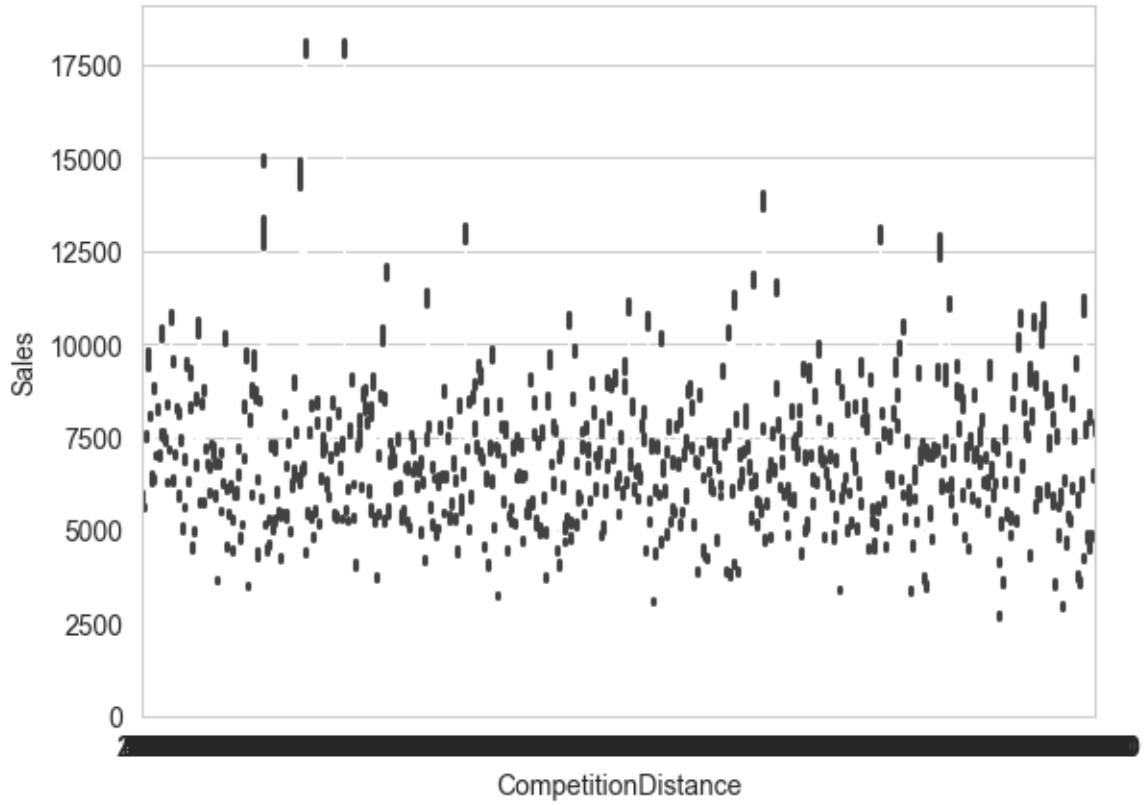

Figure 12

Direct barplot on raw CompetitionDistance — won't work as expected.

CompetitionDistance is a continuous variable with thousands of unique values, so seaborn treats each distance as a separate category.

Result: unreadable x-axis with one bar per value. Binning is needed first.

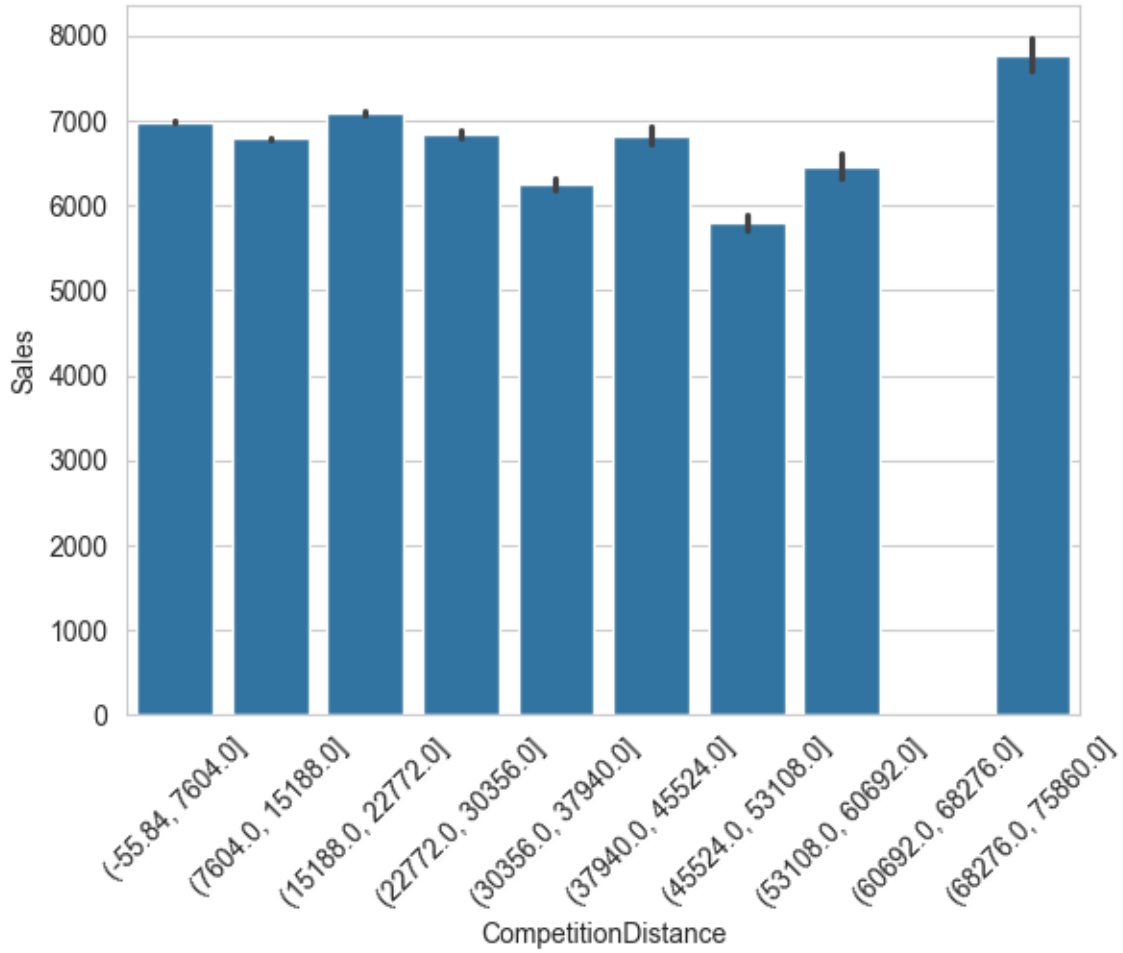

Figure 13

This bar chart, using CompetitionDistance binned into deciles, shows no strong monotonic relationship between proximity to competitors and average daily sales. Stores located very close to competitors do not consistently underperform those with more distant competition, suggesting brand loyalty and location quality override competitive proximity effects.

Figure 14

This scatter plot compares the model's predicted sales values against the true observed sales on the test set. The red dashed line represents the ideal 45-degree line of perfect prediction. The wide dispersion around this line and the narrow clustering of predicted values indicate that the linear model regresses toward the mean and cannot capture the full dynamic range of sales outcomes.

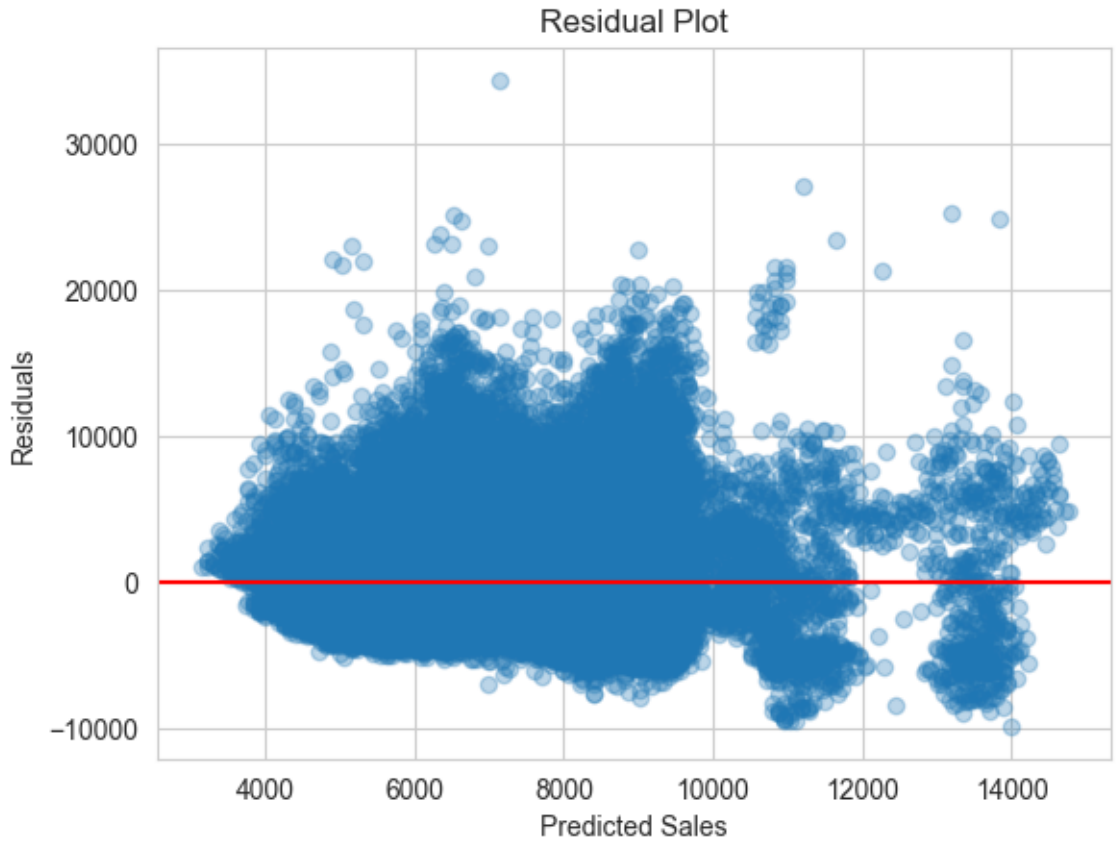

Figure 15

This plot shows the residuals (actual minus predicted sales) against the predicted values. Rather than a random, homoscedastic scatter around the red zero line, the residuals display a structured fan-shaped pattern — heteroscedasticity — where prediction errors grow larger at higher sales values. This diagnostic confirms the presence of non-linear relationships in the data that the linear model is unable to represent, strongly motivating the use of ensemble methods in future work.

10. Results & Analysis

The Linear Regression model was evaluated on the held-out test set (the latest 20% of dates). The key metrics are summarised below:

Metric	Value	Interpretation
R^2 Score	0.226	Explains ~22.6% of variance
RMSE	2.700	Average prediction error per store per day in €
MAE	1,960	Median absolute prediction error in €
MAPE	27.51%	Average % deviation from actual sales
Train Split	80% (time-based)	No data leakage from future

Predictions, residuals, and month labels were exported to model_evaluation.csv for downstream visualisation in Power BI.

The R^2 score of 0.226 indicates that the model explains approximately 22.6% of the variance in daily sales. While this is a low score in absolute terms, it is consistent with expectations for a linear model applied to complex, non-linear retail sales data.

10.1 Actual vs Predicted Plot

The scatter plot of actual versus predicted sales shows a wide dispersion around the ideal 45-degree diagonal (represented by the red dashed line). Predicted values cluster predominantly between €6,000–10,000 while actual sales vary broadly — from near zero on quiet weekdays to over €20,000 on peak promotional days. This confirms the model is unable to capture the full dynamic range of sales outcomes.

→ The tight clustering of predictions indicates that the model has regressed toward mean sales values, lacking the capacity to predict extreme high or low sales days that are driven by non-linear promotional and seasonal interactions.

10.2 Residual Plot

The residual plot (residuals vs. predicted values) reveals a clear pattern: residuals are not randomly scattered around zero but instead show a structured fan-like

spread, with larger residuals at higher predicted values. A well-fitted model would show residuals distributed randomly (homoscedastically) around the red zero line. The observed pattern (heteroscedasticity) is a diagnostic signal that the model is systematically misestimating sales in certain ranges — a hallmark of non-linear relationships that a linear model cannot represent.

10.3 Feature Importance Observations from EDA

Although Linear Regression does not provide a feature importance ranking comparable to tree-based methods, the EDA findings point to the most predictive variables:

- Promo (short-term promotion flag) — clear sales uplift, strong signal
- StoreType — Type 'b' markedly outperforms others
- Assortment — Type 'b' (extra) associated with highest sales
- DayOfWeek — strong weekly seasonality, especially Sunday drop
- Month — November–December spike is a consistent pattern
- CompetitionDistance — weak predictor, no monotonic trend

11. Conclusion

This project successfully applied a complete supervised machine learning pipeline to the Rossmann Store Sales dataset — from raw data ingestion through exploratory analysis, preprocessing, feature engineering, time-safe train-test splitting, model training, and evaluation. The Linear Regression model achieved an R^2 score of 0.226, establishing a clear and interpretable baseline for store-level daily sales prediction.

The EDA revealed a number of commercially significant trends: Store Type 'b' and Assortment 'b' are the best-performing configurations; November–December holiday seasonality is the primary annual sales driver; short-term promotions (Promo) drive real sales uplift; and continuous promotions (Promo2) show diminishing returns.

The diagnostic graphs (Actual vs. Predicted and Residuals) verify that complicated non-linear interactions between store type, promotional timing, day of week, and seasonal effects—which a linear model is essentially unable to represent—are what drive sales variance. Expected signs of this mismatch are the large prediction scatter and heteroscedastic residual pattern.

In this case, the crucial methodological choice—using a time-based rather than random train-test split—is very crucial. The model may interpolate between past and future dates with a random split, generating exaggerated assessment metrics that wouldn't translate to a real forecasting assignment.

Future research should examine ensemble techniques, especially Random Forest and XGBoost, which are excellent at capturing the interaction effects shown in EDA. Significant performance gains would probably result from additional feature innovation, such as lag features (prior week's sales), rolling averages, and store-level fixed effects. A useful standard by which these advancements might be evaluated is the R2 baseline of 0.226.